

# Project Report – MNIST Digit Classification

## 1. Introduction

- **Objective:** Build, train, and evaluate deep learning models (CNN and MLP) on the MNIST dataset.
- **Dataset:** MNIST contains 70,000 grayscale images of handwritten digits (60,000 training, 10,000 test). Each image is 28×28 pixels.
- **Goal:** Compare two models, select the best one, and deploy it in a Streamlit app for interactive predictions.

## 2. Task 1 – Dataset Loading & Exploration

- **Loaded MNIST using torchvision.datasets.MNIST.**
- Training set size: 60,000 images.
- Test set size: 10,000 images.
- Each image has shape [1, 28, 28] (grayscale channel, height, width).
- Displayed first 5 sample images with labels to verify dataset integrity.

## 3. Task 2 – Data Preprocessing

- Applied transforms:
  - `ToTensor()` → converts images to PyTorch tensors.
  - `Normalize((0.5,), (0.5,))` → scales pixel values to range [-1, 1].
- Created DataLoaders:
- Training loader: batch size = 64, shuffle = True.
- Test loader: batch size = 64, shuffle = False.

## 4. Task 3 – Model Architectures

### Model A (CNN)

- **Conv1:** 1→32 filters, kernel size 3, padding 1.
- **Conv2:** 32→64 filters, kernel size 3, padding 1.
- **Pooling:** `MaxPool2d(2,2)` after each conv.

- **FC1:** 3136→128 neurons.
- **Dropout:** 25%.
- **FC2:** 128→10 output classes.

#### Model B (MLP)

- **FC1:** 784→256 neurons.
- **Dropout:** 25%.
- **FC2:** 256→128 neurons.
- **FC3:** 128→10 output classes.

#### 5. Task 4 – Training

- Optimizer: Adam, learning rate = 0.001.
- Loss function: CrossEntropyLoss.
- Epochs: 5.
- Tracked training accuracy, validation accuracy, training loss, and validation loss per epoch.
- Plotted accuracy and loss curves (see Figures 1 & 2).

#### 6. Task 5 – Evaluation

- Compared final validation accuracy:
  - CNN achieved higher accuracy than MLP.
- Selected CNN as the best model.
- Tested on:
  - Random MNIST test images (displayed predictions vs true labels).
  - Custom handwritten digit images (PNG files uploaded).
- Displayed predictions with confidence percentages.

#### 7. Task 6 – Save & Load Model

- Saved best model weights:

```
torch.save(model.state_dict(), "mnist_model.pth")
```

- Loaded model in a new script to confirm predictions still work.

## 8. Task 7 – Streamlit App

- Built app.py to:
  - Load saved model.
  - Upload custom digit images.
  - Apply preprocessing (grayscale, resize 28×28, normalize).
  - Display predicted digit and confidence percentage.
  - Show probability distribution as a bar chart.
- Added demo section to test 5 random MNIST images.

## 9. Results

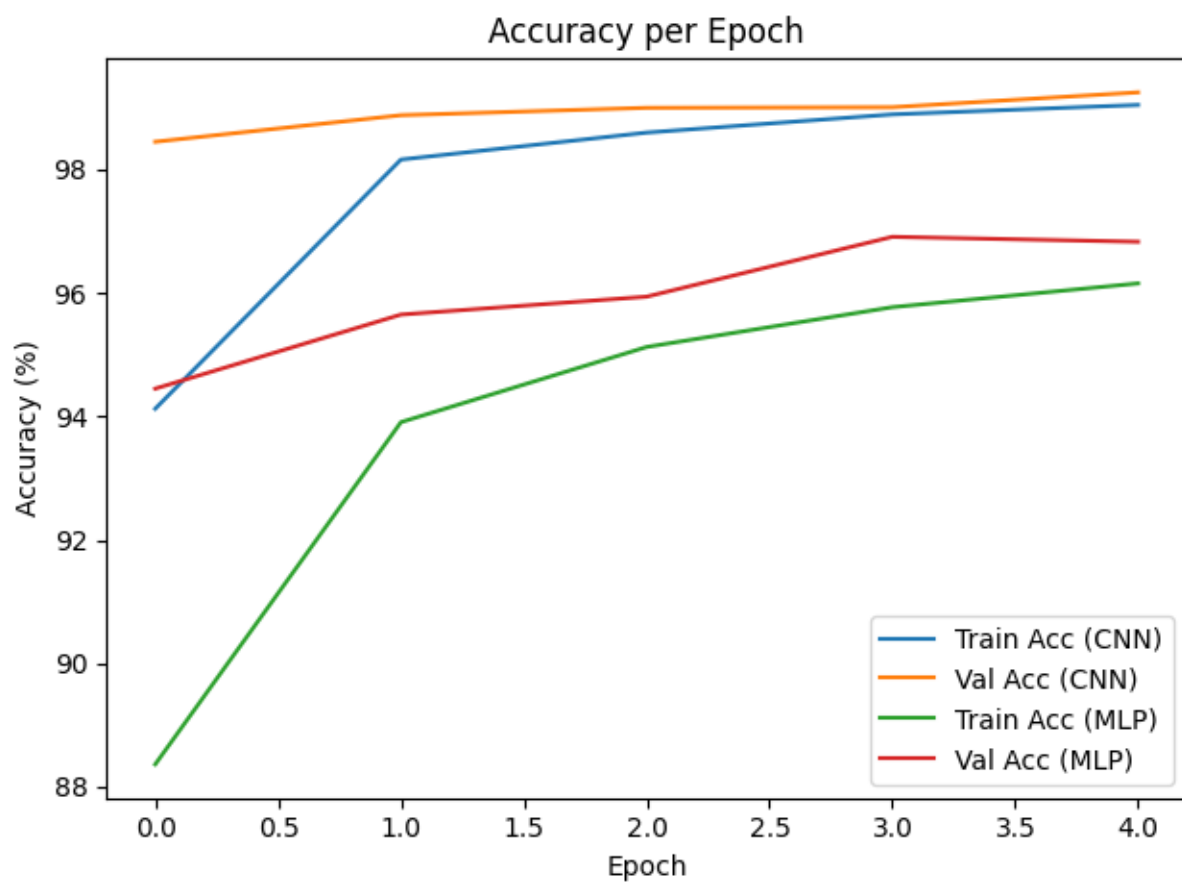
- **CNN** consistently outperformed MLP on MNIST.
- Accuracy curves showed CNN converged faster and achieved higher validation accuracy.
- Streamlit app successfully predicts digits from both MNIST and custom images.
- Confidence percentages provide interpretability of predictions.

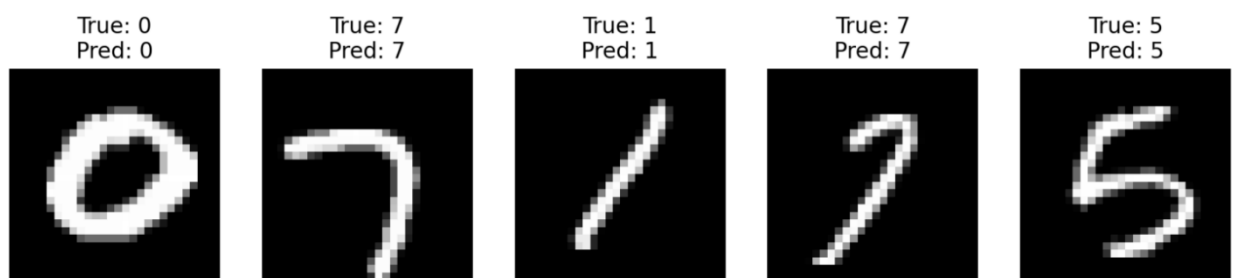
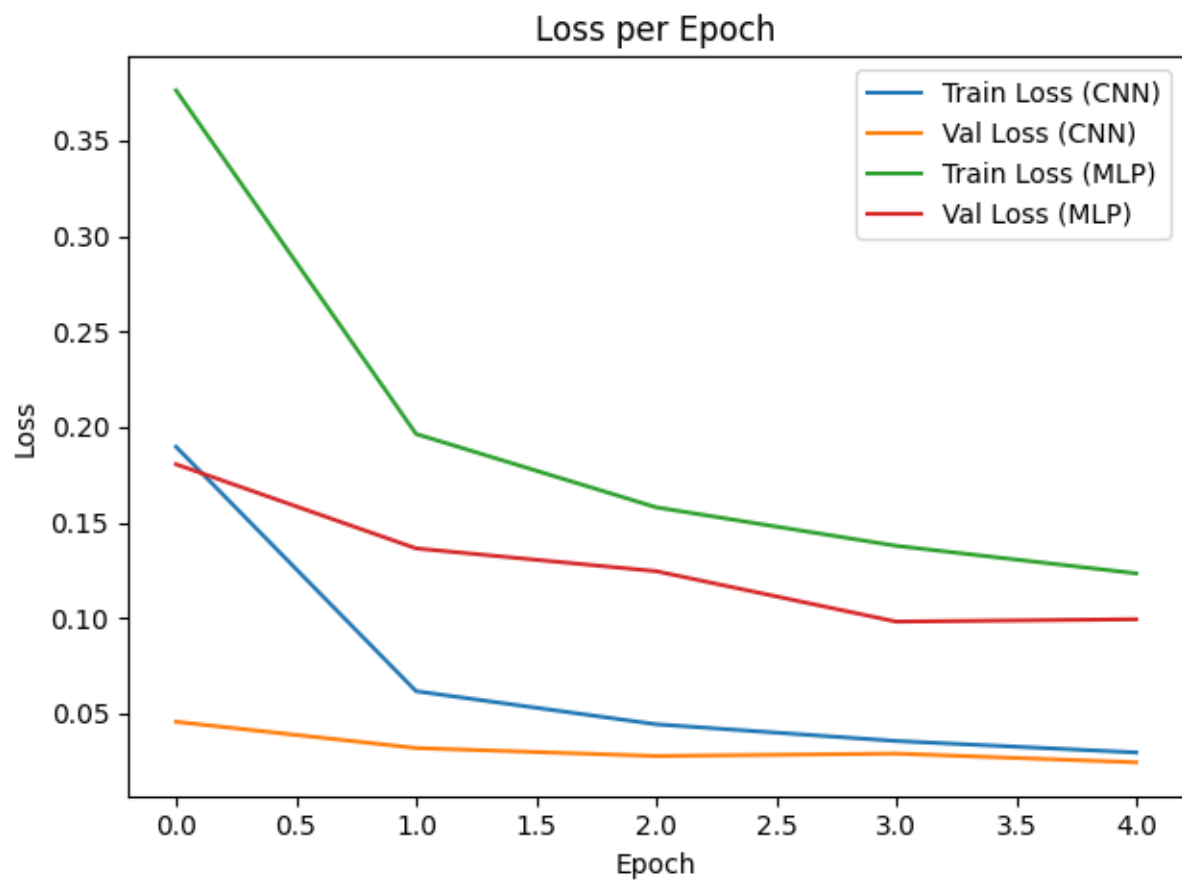
## 10. Conclusion

- CNN is the best model for MNIST digit classification.
- The project demonstrates the full pipeline: dataset loading, preprocessing, model building, training, evaluation, saving/loading, and deployment in a Streamlit app.
- Future improvements: more epochs, data augmentation, learning rate scheduling.

## Figures

- **Figure 1:** Accuracy per epoch (CNN vs MLP).
- **Figure 2:** Loss per epoch (CNN vs MLP).
- **Figure 3:** Predictions on random MNIST test images.
- **Figure 4:** Predictions on custom handwritten digits.





**Figure 3:** Predictions on random MNIST test images.



**Figure 4:** Predictions on custom handwritten digits.